

Construction d'un perceptron

Alizée MESNARD

14.04.2025

1 Perceptron unique sur des données linéairement séparables

1.1 Génération des données

La première base de données a été générée à l'aide du code suivant :

```
1 from sklearn.datasets import make_blobs
2 X, y = make_blobs(n_samples=300, n_features=2, centers=2, random_state=0)
```

Ce code génère un jeu de données contenant **300 observations** réparties en **2 classes**. Les données sont générées de manière à être linéairement séparables.

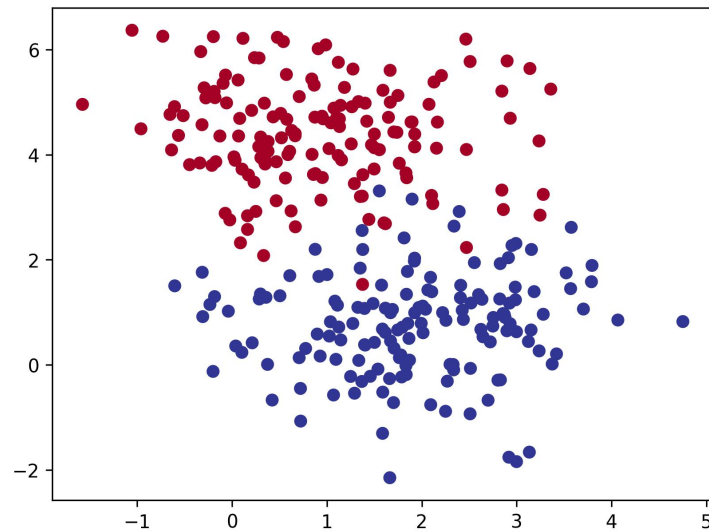


FIGURE 1 – Visualisation des données linéairement séparables générées par make_blobs

1.2 Implémentation du perceptron

L'implémentation du perceptron est directement basée sur les formules mathématiques ci-dessous :

1.2.1 Le modèle du perceptron

Le perceptron est défini par l'équation :

$$z(x) = w_1x_1 + w_2x_2 + \dots + w_px_p + b \quad (1)$$

$$= \sum_{i=1}^p w_i x_i + b \quad (2)$$

Avec une fonction d'activation (sigmoïde dans notre cas) :

$$a(x) = \sigma(z(x)) = \frac{1}{1 + e^{-z(x)}} \quad (3)$$

C'est exactement ce qui est implémenté dans la fonction `model()` :

```

1 def model(X, W, b):
2     Z = X.dot(W) + b    # Z = WX^T + b
3     A = 1 / (1 + np.exp(-Z)) # Activation sigmoid
4     return A

```

1.2.2 La fonction de coût (entropie croisée binaire)

$$l(y, \hat{y}) = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})) \quad (4)$$

Cette fonction dérive de la maximisation de la vraisemblance. Le risque empirique est alors défini comme :

$$\mathcal{R}(\theta) = \frac{1}{N} \sum_{i=1}^N l(y^{(i)}, f_{\theta}(X^{(i)})) \quad (5)$$

Mon implémentation dans `log_loss()` suit exactement cette formule :

```

1 def log_loss(A, y):
2     return 1 / len(y) * np.sum(-y * np.log(A) - (1 - y) * np.log(1 - A))

```

1.2.3 Calcul des gradients

L'approche de la descente de gradient est décrite avec les formules des dérivées partielles :

$$\frac{\partial \mathcal{R}}{\partial w_k} = -\frac{1}{N} \sum_{i=1}^N (y^{(i)} - a^{(i)}) x_k^{(i)} \quad (6)$$

$$\frac{\partial \mathcal{R}}{\partial b} = -\frac{1}{N} \sum_{i=1}^N (y^{(i)} - a^{(i)}) \quad (7)$$

La fonction `gradients()` implémente directement ces formules (avec une inversion du signe due à la minimisation plutôt que la maximisation) :

```

1 def gradients(A, X, y):
2     dW = 1 / len(y) * np.dot(X.T, A - y)
3     db = 1 / len(y) * np.sum(A - y)
4     return (dW, db)

```

1.2.4 Mise à jour des paramètres

Le cours présente la règle de mise à jour de la descente de gradient :

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{R}_{\theta} \quad (8)$$

Ce qui se traduit dans la fonction `update()` :

```

1 def update(dW, db, W, b, learning_rate):
2     W = W - learning_rate * dW
3     b = b - learning_rate * db
4     return (W, b)

```

1.3 Architecture complète du perceptron

J'ai développé les fonctions suivantes pour construire mon perceptron :

- `initialisation(X)` : Crée des poids W et un biais b initiaux aléatoires pour un réseau de neurones basé sur les dimensions des données d'entrée.
- `model(X, W, b)` : Calcule la propagation avant en calculant $Z = WX^T + b$ et en appliquant la fonction d'activation sigmoïde.
- `log_loss(A, y)` : Calcule la perte d'entropie croisée binaire entre les probabilités prédites A et les étiquettes réelles y .

- `gradients(A, X, y)` : Calcule les gradients de la fonction de perte par rapport aux poids W et au biais b .
- `update(dW, db, W, b, learning_rate)` : Met à jour les paramètres du modèle (poids et biais) en utilisant la descente de gradient.
- `predict(X, W, b)` : Effectue des prédictions binaires en appliquant le modèle et en seuillant à 0,5.
- `artificial_neuron(X, y, learning_rate, n_iter)` : Implémente un processus complet d'entraînement du perceptron, comprenant l'initialisation, les itérations d'entraînement et l'évaluation.

1.4 Résultats

J'ai ensuite appliqué ce modèle au jeu de données et obtenu une frontière de décision linéaire qui sépare correctement les deux classes. J'ai visualisé cette frontière et évalué la précision du modèle. Le perceptron a obtenu d'excellentes performances sur ce jeu de données linéairement séparable, atteignant une précision proche de 100%.

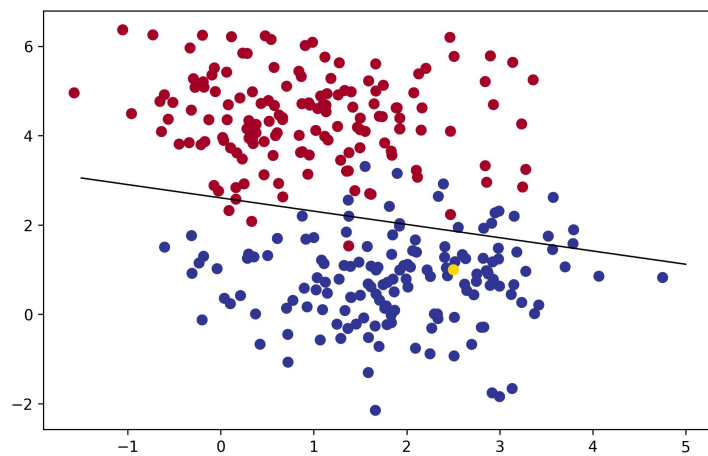


FIGURE 2 – Frontière de décision linéaire générée par le perceptron

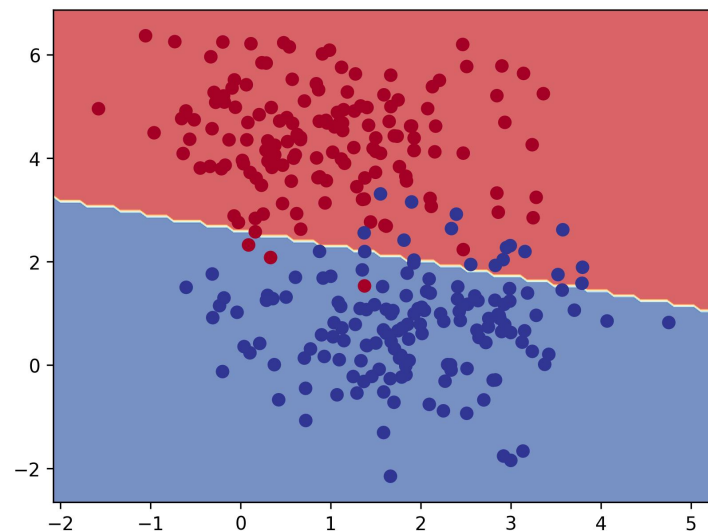


FIGURE 3 – Visualisation des régions de décision pour les données linéaires

2 Perceptron sur des données circulaires

2.1 Génération des données

J'ai également utilisé une autre base de données, générée avec le code suivant :

```
1 from sklearn.datasets import make_circles
2 X_circ, y_circ = make_circles(n_samples=500, noise=0.1, factor=0.3, random_state=0)
```

Ce code génère un jeu de données où les points sont disposés en deux cercles concentriques, ce qui représente un problème de classification non linéairement séparable. Chaque cercle correspond à une classe différente.

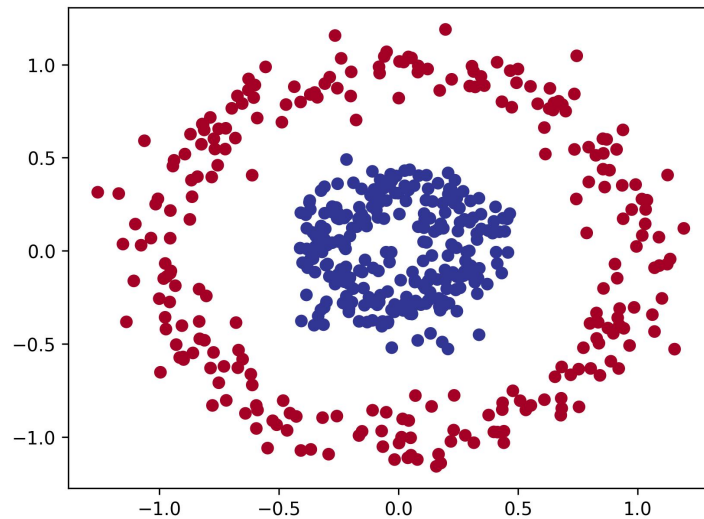


FIGURE 4 – Visualisation des données circulaires générées par `make_circles`

2.2 Application du perceptron et résultats

J'ai appliqué le même perceptron à ce nouveau jeu de données et visualisé la frontière de décision obtenue. Comme prévu, le perceptron n'a pas été capable de séparer correctement les deux classes, car il ne peut produire qu'une frontière de décision linéaire. Les performances du modèle ont été significativement inférieures sur ce jeu de données, avec une précision d'environ 50% (équivalente à un classement aléatoire).

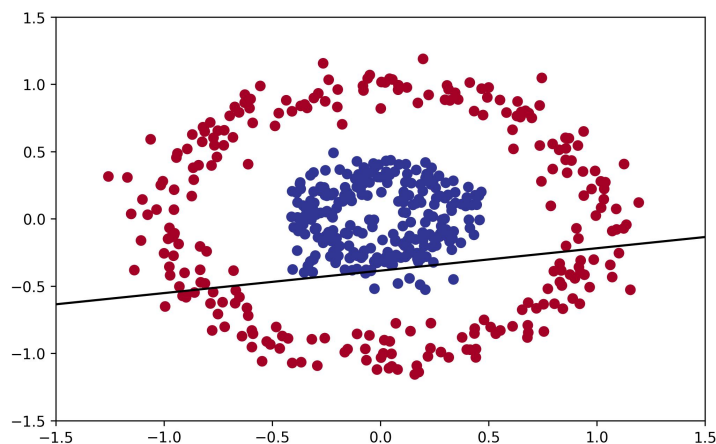


FIGURE 5 – Frontière de décision linéaire inadaptée aux données circulaires

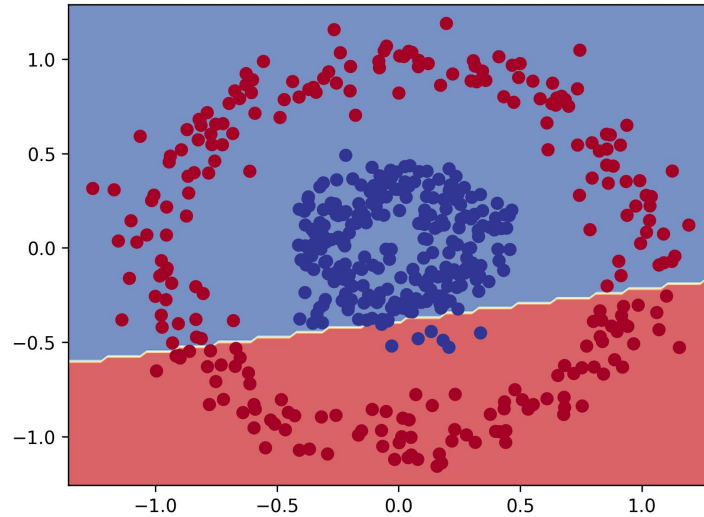


FIGURE 6 – Visualisation des régions de décision pour les données circulaires

2.3 Limitations et lien avec la théorie

Cette expérience a clairement démontré la limitation fondamentale du perceptron : son incapacité à résoudre des problèmes non linéairement séparables. Ce constat illustre parfaitement ce qui est expliqué dans le cours : le perceptron simple définit une frontière de décision linéaire de la forme $z(x) = 0$ qui correspond à $\sum_{i=1}^p w_i x_i + b = 0$.

Pour des données comme les cercles concentriques, aucune droite ne peut séparer correctement les deux classes. Il faudra des perceptrons multicouches, qui en combinant plusieurs neurones avec des fonctions d'activation non linéaires, peuvent approximer des frontières de décision non linéaires.

Cette observation est cohérente avec les limites théoriques du perceptron et démontre l'importance du théorème d'approximation universelle, qui stipule qu'un perceptron multicouche avec suffisamment de neurones peut approximer n'importe quelle fonction continue sur un ensemble compact.